

Dokumentacja Biblioteki PUTM EV CAN

Michał Błotniak

1 kwietnia 2026

Streszczenie

Niniejszy dokument stanowi instrukcję obsługi biblioteki `PUTM_EV_CAN_LIBRARY`. Biblioteka została zaprojektowana dla systemów wbudowanych (STM32) oraz środowiska ROS2. Główne cechy to: statyczna alokacja pamięci, obsługa wielu rodzin STM32 (F4, G4, L4, H7) dzięki warstwie abstrakcji HAL, odbiór danych oparty na przerwaniach (Interrupt-Driven). Dokument zawiera również zbiór dobrych praktyk oraz najczęstszych pułapek sprzętowo-programowych.

Legenda kolorów: STM32 ROS 2 / Linux Wspólne

Spis treści

1	Wymagania systemowe i zależności	2
1.1	Instalacja zależności Python	2
2	Przygotowanie Plików DBC	3
3	Integracja i Konfiguracja w CMake	4
4	Konwencja Nomenklatury Magistral	6
5	Przygotowanie Sprzętowe (STM32CubeMX)	7
6	Poprawna Inicjalizacja Oprogramowania	9
7	Wysyłanie (TX) i Odbieranie danych (RX)	11
8	Rozwiązywanie Problemów i Znane Pułapki (FAQ)	13

1 Wymagania systemowe i zależności

Prawidłowa kompilacja i działanie biblioteki PUTM_EV_CAN_LIBRARY wymaga zapewnienia odpowiedniego środowiska programistycznego. Poniższa tabela przedstawia minimalne wersje narzędzi oraz bibliotek wykorzystywanych w projekcie.

Narzędzie / Biblioteka	Wersja Minimalna	Uwagi
CMake	3.16	Wymagane przez konfigurację projektu.
Standard C++	C++20	Wymuszone w pliku CMakeLists.txt .
Standard C	C11	Standard dla kodu C.
Python	3.8+	Niezbędny do uruchomienia skryptów generujących kod z DBC.
Kompilator C++ (ARM)	GCC $\geq$ 10.0	Musi obsługiwać flagi standardu C++20.
Biblioteka HAL	STM32XXxx HAL	Zależność sprzętowa dla mikrokontrolerów różnych serii.
Python: cantools	40.7.1	Generator kodu C z plików DBC.
Python: python-can	(auto)	Instalowana automatycznie jako zależność cantools .

Tabela 1: Wymagania systemowe projektu

1.1 Instalacja zależności Python

Skrypt generujący kod (`dbc/setup.py`) jest uruchamiany automatycznie przez CMake i wymaga zainstalowanych bibliotek w środowisku Pythona. Brak tych bibliotek spowoduje błąd konfiguracji CMake na etapie **Checking and regenerating DBC files**.

```
1 pip install cantools==40.7.1
```

Listing 1: Instalacja wymaganych bibliotek Python

2 Przygotowanie Plików DBC

Biblioteka generuje kod C na podstawie plików .dbc opisujących sygnały magistral CAN. **Pliki te są przechowywane w osobnym repozytorium i muszą zostać ręcznie skopiowane do odpowiedniego katalogu projektu przed konfiguracją CMake.**

Krok 1: Pobranie plików DBC

Aktualne pliki .dbc znajdują się w dedykowanym repozytorium:

https://github.com/PUT-Motorsport/PUTM_EV_CAN_DBC

WAŻNE: Zawsze pobieraj pliki z gałęzi main, z oznaczeniem najnowszej wersji - postram się, aby to zawsze to był ostatni commit, ale proszę mieć na uwadze sprawdzanie na jakiej aktualnie konfiguracji pracuje magistrala. Użycie nieaktualnych plików może skutkować niezgodnością identyfikatorów ramek i nazw sygnałów z resztą systemu, lub doprowadzać do zakłamania wysłanych / odbieranych danych.

Krok 2: Umieszczenie plików w projekcie

Pobrane pliki .dbc należy umieścić w katalogu dbc/ wewnątrz biblioteki:

```
1 PUTM_EV_CAN_LIBRARY/  
2     database/  
3         dbc/          <- TUTAJ umieszczamy pliki .dbc  
4         .gitkeep      <- plik konfiguracyjny dla git
```

CMake automatycznie wykryje pliki .dbc i uruchomi skrypt generujący kod C podczas kolejnej konfiguracji projektu.

UWAGA

Zakaz wykonywania w tych folderach jakichkolwiek operacji push. Może to doprowadzić do zaburzenia struktury pracy na git.

3 Integracja i Konfiguracja w CMake

Biblioteka jest dystrybuowana jako moduł CMake. Aby użyć jej w projekcie, należy zmodyfikować główny plik CMakeLists.txt projektu.

STM32

Integracja w projekcie STM32

Integracja biblioteki PUTM_EV_CAN_LIBRARY wymaga precyzyjnej modyfikacji głównego pliku CMakeLists.txt Twojego projektu. Nawet drobne braki w tym pliku prowadzą do trudnych w diagnozie błędów kompilacji. Poniżej przedstawiono 4 obowiązkowe kroki poprawnej konfiguracji. **BARDZO WAŻNE:** CMake jest niezwykle rygorystyczny w kwestii słów kluczowych określających zasięg (np. PRIVATE, PUBLIC). Należy stosować je konsekwentnie we wszystkich modyfikowanych sekcjach.

Krok 1: Dodanie podkatalogu biblioteki

Należy poinformować CMake, gdzie fizycznie znajdują się pliki źródłowe nowej biblioteki. Dodajemy tę instrukcję zazwyczaj pod włączeniem źródeł wygenerowanych przez CubeMX.

```
1 # Dodanie biblioteki CAN (upewnij się, że ścieżka jest poprawna)
2 add_subdirectory(${PROJECT_SOURCE_DIR}/Components/PUTM_EV_CAN_LIBRARY)
```

Krok 2: Konfiguracja ścieżek nagłówkowych (Include Directories)

Zewnętrzne projekty odnoszą się do plików biblioteki przy użyciu pełnej ścieżki, np. #include "PUTM_EV_CAN_LIBRARY/include/stm32_hal_selector.hpp". Aby kompilator mógł to odnaleźć, musisz dodać do ścieżek poszukiwań **katalog nadrzędny** biblioteki (w tym przypadku folder Components).

Częsty błąd: Brak tej ścieżki skutkuje błędem: fatal error: PUTM_EV_CAN_LIBRARY/...: No such file or directory.

Przykład:

```
1 # Add include paths
2 target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
3     "${PROJECT_SOURCE_DIR}/Core/Inc"
4     "${PROJECT_SOURCE_DIR}/Components" # <- KATALOG NADRZĘDNY BIBLIOTEKI (
5     WYMAGANE!)
6     "${PROJECT_SOURCE_DIR}/Components/PUTM_EV_CAN_LIBRARY/include"
7     "${PROJECT_SOURCE_DIR}/Components/PUTM_EV_CAN_LIBRARY/dbc/generated"
```

Krok 3: Linkowanie biblioteki

Kompilator musi fizycznie dołączyć skompilowany kod biblioteki (.a lub pliki obiektowe) do Twojego pliku wynikowego (.elf). Robimy to w sekcji target_link_libraries.

Częsty błąd: Brak linkowania spowoduje błędy na samym końcu budowania (faza linkera): undefined reference to 'can_driver'.

```
1 # Add linked libraries
2 target_link_libraries(${CMAKE_PROJECT_NAME}
3     stm32cubemx
4     putm_ev_can # <- DODANIE ZBUDOWANEJ BIBLIOTEKI CAN
5 )
```

Wspólne (STM32 i ROS 2)

Wybór konfiguracji (STM32 vs ROS2)

System CMake automatycznie próbuje wykryć środowisko (Bare-metal STM32 vs system Linux z załadowanym ROS2). Jeśli autodetekcja zawiedzie, możesz wymusić zachowanie ręcznie.

```
1 # Dla STM32:
2 set(PUTM_CAN_BACKEND "STM32" CACHE STRING "Select backend" FORCE)
3
4 # Dla ROS2 / Linux:
5 set(PUTM_CAN_BACKEND "ROS2" CACHE STRING "Select backend" FORCE)
```

Listing 2: Wymuszanie konfiguracji w CMake

ROS 2 / Linux

Integracja w pakiecie ROS 2 (Linux)

Korzystanie z biblioteki w środowisku ROS 2 wymaga pewnych zmian wynikających z ekosystemu Ament.

Krok 1: Wymuszenie standardu C++20

ROS 2 Humble domyślnie używa standardu C++17. Posiadanie nowej biblioteki opartej na nowoczesnych typach (jak `std::span`) wymusza zmianę standardu bezpośrednio pod deklaracją projektu.

```
1 project(putm_vcl)
2 set(CMAKE_CXX_STANDARD 20) # KONIECZNE DLA ROS 2
```

Krok 2: CMake dla *PUTM_VCL*

Poniżej przedstawiono wzór poprawnej konfiguracji pliku `CMakeLists.txt` dla pakietu `putm_vcl`. Docelowo krok ten powinien być już zaimplementowany w głównym repozytorium. Opis ten ma jednak kluczowe znaczenie w przypadku tworzenia nowej gałęzi (*brancha*) od zera lub konieczności ponownej integracji biblioteki z węzłami ROS 2.

```
1 set(PUTM_CAN_BACKEND "ROS2" CACHE STRING "Select backend" FORCE)
2 add_subdirectory(include/PUTM_EV_CAN_LIBRARY putm_ev_can_build)
3
4 # ... deklaracje targetow ROS (add_executable) ...
5
6 ament_target_dependencies(can_tx_node rclcpp putm_vcl_interfaces)
7
8 target_link_libraries(can_tx_node putm_ev_can)
9 target_link_libraries(can_rx_node putm_ev_can)
```

4 Konwencja Nomenklatury Magistral

W projekcie pojazdu zdefiniowano trzy fizyczne magistrale CAN. Aby zachować spójność w kodzie, **należy bezwzględnie** stosować poniższe nazewnictwo instancji klasy sterownika (np. **CanDriver** lub **AppCAN** zależnie od wersji wrappera).

Nazwa Instancji	Funkcja Magistrali	Opis
can_m	Main	Główna magistrala, większość danych, telemetria.
can_pt	Powertrain (ECU/INV)	Komunikacja wyłącznie falownik - silniki.
can_dv	Driverless	Wymiana sygnałów w trybie jazdy autonomicznej.

Tabela 2: Obowiązująca nomenklatura instancji

5 Przygotowanie Sprzętowe (STM32CubeMX)

STM32

ZAKAZ: Nie zostawiaj domyślnych ustawień dla FDCAN w CubeMX.

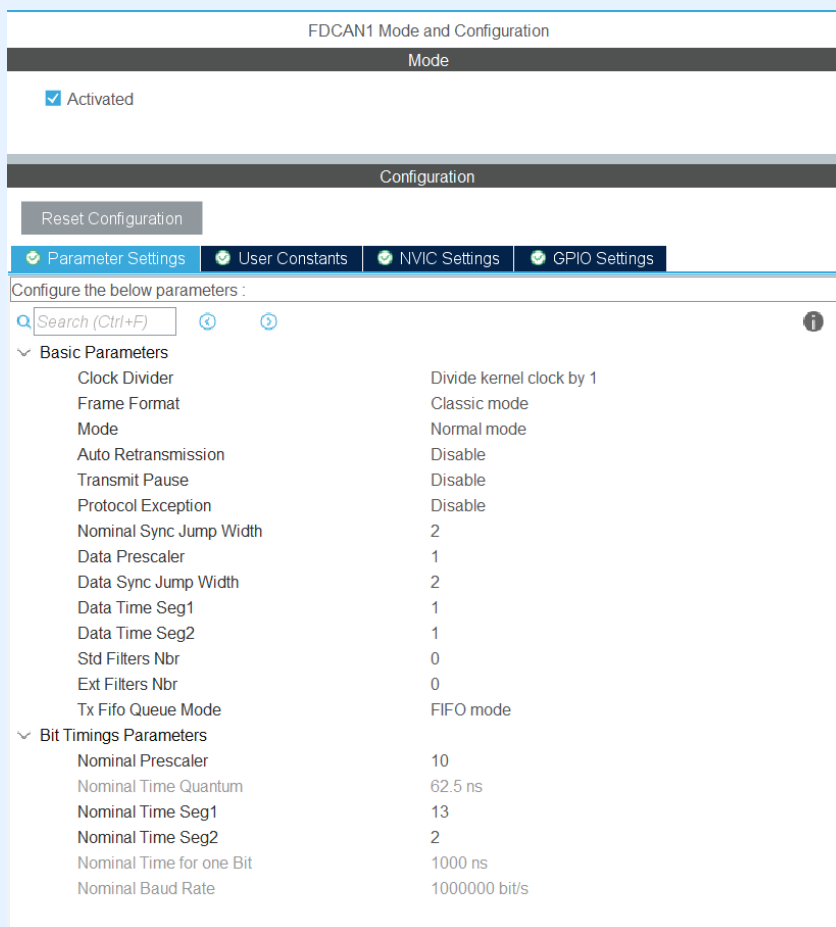
Krok 1: Parameter Settings (Parametry FDCAN)

W zakładce FDCAN1/FDCAN2 → Parameter Settings należy bezwzględnie ustawić:

- **Tx Fifo Queue Mode:** FIFO mode .
- **Frame Format:** Classic Mode
- **Mode:** Normal Mode
- **Auto Retransmission:** Disable. Włączenie tej funkcji może doprowadzić do zapchania magistrali.

Do ustawienia poniższych wartości należy użyć skryptu znajdującego się w folderze z dokumentacją „*setup_STM32.py*”. Po wprowadzeniu konfiguracji zegarów skrypt zwróci wartości jakie trzeba wpisać w pola Nominal Prescaler, Nominal Time Seg1 oraz Nominal Time Seg2, aby uzyskać parametry opisane poniżej.

- **Nominal Baud Rate** musi wynosić $1 \text{ Mb/s} = 1\,000\,000 \text{ bit/s}$.
- **Nominal Time Quantum** powinna być w okolicach 60-80.



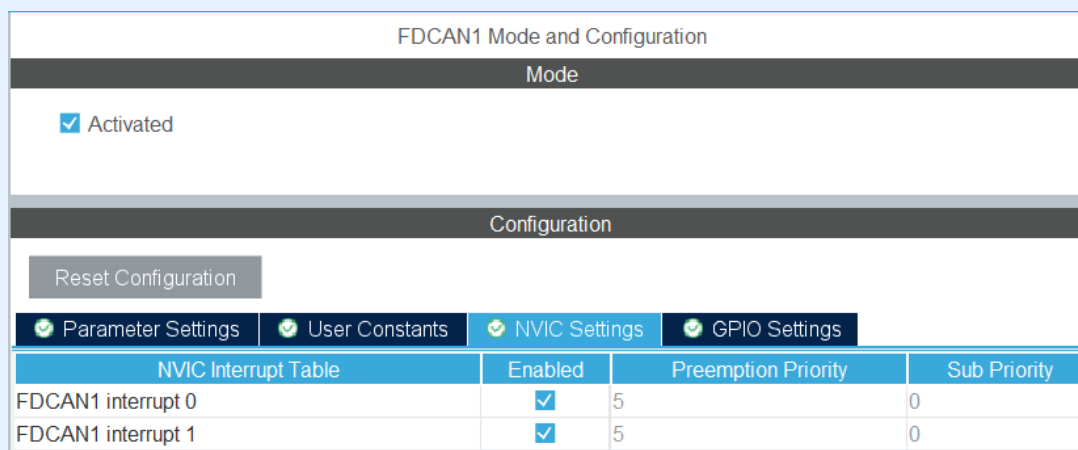
FDCAN1 Mode and Configuration	
Mode	
☒ Activated	
Configuration	
Reset Configuration	
Parameter Settings User Constants NVIC Settings GPIO Settings	
Configure the below parameters :	
Search (Ctrl+F)	
Basic Parameters	
Clock Divider	Divide kernel clock by 1
Frame Format	Classic mode
Mode	Normal mode
Auto Retransmission	Disable
Transmit Pause	Disable
Protocol Exception	Disable
Nominal Sync Jump Width	2
Data Prescaler	1
Data Sync Jump Width	2
Data Time Seg1	1
Data Time Seg2	1
Std Filters Nbr	0
Ext Filters Nbr	0
Tx Fifo Queue Mode	FIFO mode
Bit Timings Parameters	
Nominal Prescaler	10
Nominal Time Quantum	62.5 ns
Nominal Time Seg1	13
Nominal Time Seg2	2
Nominal Time for one Bit	1000 ns
Nominal Baud Rate	1000000 bit/s

Rysunek 1: Zalecane ustawienia parametrów FDCAN w STM32CubeMX.

Krok 2: NVIC Settings (Przerwania)

Aby funkcja asynchronicznego odbioru (Interrupt-Driven) dostarczana przez bibliotekę działała poprawnie, musisz aktywować odpowiednie przerwania w zakładce *NVIC Settings*:

- Zaznacz **Enabled** dla FDCAN1 interrupt 0.
- Zaznacz **Enabled** dla FDCAN1 interrupt 1.

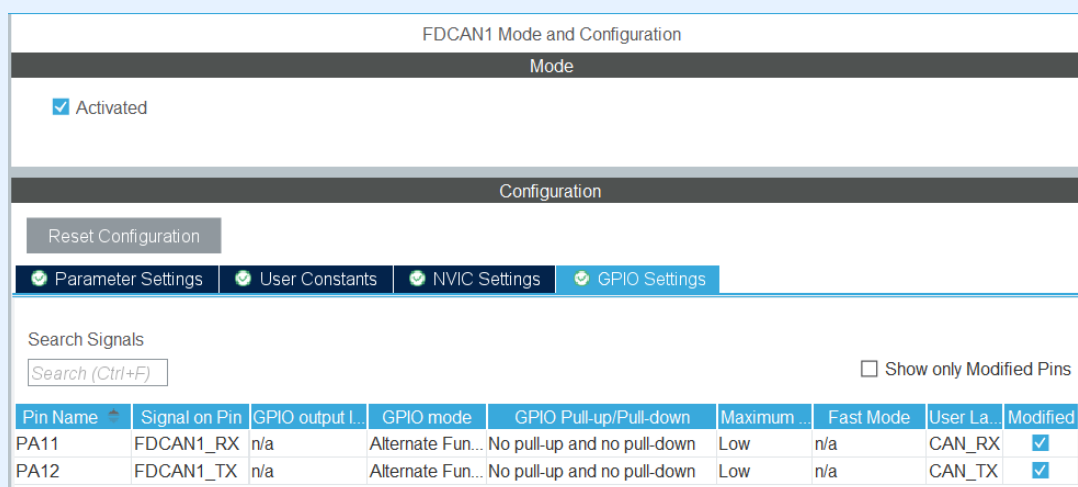


Rysunek 2: Konfiguracja przerwań NVIC dla magistrali FDCAN.

Krok 3: GPIO Settings (Piny układu)

Ostatnim krokiem jest upewnienie się, że odpowiednie piny mikrokontrolera zostały przypisane do funkcji RX i TX magistrali CAN w zakładce *GPIO Settings*:

- Przypisz sygnały FDCAN1_RX oraz FDCAN1_TX do właściwych pinów (np. odpowiednio PA11 i PA12).
- Zaleca się dodanie własnych etykiet (User Label), np. CAN_RX i CAN_TX, co znacznie poprawia czytelność wygenerowanego kodu.



Rysunek 3: Przypisanie pinów GPIO dla FDCAN.

6 Poprawna Inicjalizacja Oprogramowania

Biblioteka przejmuje całkowitą kontrolę nad peryferium CAN. Inicjalizuje filtry, aktywuje powiadomienia o przerwaniach i fizycznie uruchamia magistralę.

STM32

Inicjalizacja na STM32

Krok 1: Instancje Globalne

W pliku `main.cpp` (lub pliku obsługującym konkretne zadanie FreeRTOS, np. `communication_task.cpp`) należy utworzyć obiekty dla wykorzystywanych magistral bez słowa `extern`, jeśli definicja znajduje się w tym samym pliku.

```
1 // Wlasciwa definicja obiektu w pliku .cpp
2 putm_ev_can::CanDriver can_m;
```

Krok 2: Wywołanie `Init()` i Ciche Błędy

ZAKAZ PODWÓJNEJ INICJALIZACJI: Nie wolno wywoływać funkcji `HAL_FDCAN_Start()` oraz `HAL_FDCAN_ActivateNotification()` z poziomu pliku `main.c`! Biblioteka wewnątrz metody `can_m.Init(&hfdcan1)` sama to robi. Jeśli wywołasz `Start` sprzętowy przed biblioteką, funkcja `Init()` zwróci błąd (status *BUSY*), flaga wewnętrzna biblioteki pozostanie w stanie *uninitialized*, a funkcja `Send()` nigdy nie wyśle ramki.

Dobra Praktyka (Poprawny kod):

```
1 /* USER CODE BEGIN 2 */
2 // Tylko konfiguracja filtrow (jesli potrzebna)
3 FDCAN_FilterTypeDef filter_config;
4 // ... ustawienia filtra ...
5 HAL_FDCAN_ConfigFilter(&hfdcan1, &filter_config);
6
7 // ZADNYCH HAL_FDCAN_Start() TUTAJ! Zostaw to bibliotece.
8 /* USER CODE END 2 */
9
10 // ... pozniej, w tasku lub przed petla glowna ...
11 void Communication_Task(void* argument) {
12     // 1. To wystarczy do uruchomienia komunikacji
13     if (!can_m.Init(&hfdcan1)) {
14         // Bład inicjalizacji (np. zablokowany sprzet)
15         Error_Handler();
16     }
17     // ...
18 }
```

Inicjalizacja w ROS 2 (Linux)

W środowisku opartym na SocketCAN, do funkcji `Init()` przekazujemy nazwę logiczną interfejsu sieciowego (np. "can0"). Zazwyczaj robimy to bezpośrednio w konstruktorze węzła ROS 2.

```
1 #include "can_driver.hpp"
2
3 class CanRxNode : public rclcpp::Node {
4 private:
5     putm_ev_can::CanDriver can_amk;
6 public:
7     CanRxNode() : Node("can_rx_node") {
8         if (!can_amk.Init("can0")) {
9             RCLCPP_ERROR(this->get_logger(), "Failed to init AMK CAN interface"
10         );
11     }
12 };
```

7 Wysyłanie (TX) i Odbieranie danych (RX)

STM32

Odbieranie poprzez Callbacki na STM32 (Interrupt-Driven)

Odbiór w nowej wersji biblioteki działa asynchronicznie.

UWAGA: Kod wewnątrz callbacku wykonuje się w **KONTEKŚCIE PRZERWANIA SPRZĘTOWEGO (ISR)**.

ZAKAZY wewnątrz Callbacku:

- Nie używaj funkcji blokujących (np. `osDelay`, `HAL_Delay`).
- Nie wykonuj ciężkich obliczeń matematycznych.
- Nie wywołuj funkcji typu `printf`.

Dobra Praktyka: Zmieniaj stan zmiennych oznaczonych jako `volatile`, odświeżaj watchdoga lub używaj `osMutexAcquire(..., 0)` (czas oczekiwania 0 dla przerwania!).

```
1 // Rejestracja musi odbyć się tylko raz!
2 can_m.RegisterCallback<PUTM_CAN_M_front_data_t>(<
3     PUTM_CAN_M_FRONT_DATA_FRAME_ID,
4     [] (const PUTM_CAN_M_front_data_t& front_data) {
5
6         HAL_IWDG_Refresh(&hiwdg); // Szybka, dozwolona operacja w ISR
7
8         // Blokada mutexu z czasem = 0 (aby nie zablokować przerwania)
9         if (osMutexAcquire(dataMutexHandle, 0) == osOK) {
10             data.brake_light = front_data.is_braking;
11             osMutexRelease(dataMutexHandle);
12         }
13     }
14 );
```

Listing 3: Rejestracja callbacku i bezpieczne operacje na STM32

Wysyłanie Danych (STM32)

Metoda `Send` sprawdza typ i bezpiecznie umieszcza ramkę w kolejce (dlatego konfiguracja FIFO w CubeMX była tak kluczowa).

```
1 // Przykład w petli for(;;)
2 PUTM_CAN_M_rearbox_temperature_t temp_msg = {
3     .mono_temperature = (uint8_t)temperature.mono,
4     .coolant_temperature_in = (uint8_t)temperature.coolant_in,
5     .coolant_temperature_out = (uint8_t)temperature.coolant_out,
6     .oil_temperature_l = (uint8_t)temperature.oil_l,
7     .oil_temperature_r = (uint8_t)temperature.oil_r
8 };
9
10 // Dobra praktyka jest weryfikacja czy ramka została dodana do kolejki
11 if (!can_m.Send(PUTM_CAN_M_REARBOX_TEMPERATURE_FRAME_ID, temp_msg)) {
12     // np. zapal czerwona dioda błędu na uk adzie TCA.
13 }
```

Odbieranie i wysyłanie w ROS 2 (Thread Safety)

W systemie Linux implementacja SocketCAN wywołuje zarejestrowane lambdy z wewnętrznego wątku pracującego w tle (`std::thread rx_thread_loop`).

W przeciwieństwie do STM32, **nie jesteśmy tutaj w przerwaniu sprzętowym**. Dodatkowo, publishery w ROS 2 (`rclcpp::Publisher::publish`) są **domyślnie bezpieczne wątkowo (thread-safe)**, więc publikowanie wiadomości bezpośrednio z wnętrza callbacku CAN jest w pełni bezpieczne i wysoce zalecane.

```
1 #include "PUTM_CAN_M.h"
2
3 // Rejestracja w konstruktorze wezla
4 can_common.RegisterCallback<PUTM_CAN_M_driver_input_t>(
5     PUTM_CAN_M_DRIVER_INPUT_FRAME_ID,
6     [this](const PUTM_CAN_M_driver_input_t& frame) {
7
8         // Zbuduj wiadomosc wysokopoziomowa ROS 2
9         msg::FrontboxDriverInput ros_msg;
10        ros_msg.pedal_position = frame.pedal_position;
11
12        // Bezpieczna publikacja bezposrednio z watku w tle
13        frontbox_driver_input_publisher->publish(ros_msg);
14    }
15 );
```

Listing 4: Przykład asynchronicznego odbioru w ROS 2

8 Rozwiązywanie Problemów i Znane Pułapki (FAQ)

Poniżej zestawiono najczęściej spotykane problemy, objawy oraz rozwiązania sprzętowo-programowe.

STM32

Pułapki Software'owe (STM32)

- **Objaw:** Kod się wykonuje, ale ramki nie wychodzą na zewnątrz (`Send` zwraca błąd).
 - **Przyczyna 1:** W CubeMX nie zdefiniowano rozmiaru kolejek FDCAN (`Tx FIFO/Queue Elmts Nbr` równe 0).
 - **Przyczyna 2:** Przedwczesne uruchomienie sprzętu przez `HAL_FDCAN_Start()` przed zainicjowaniem biblioteki.
- **Objaw:** Błąd w CMake "Backend not defined".
 - **Przyczyna:** Brak podlinkowania biblioteki do targetu przez `target_link_libraries(... PRIVATE putm_ev_can)`.

STM32

Pułapki Sprzętowe (Hardware)

- **Objaw:** Brak sygnału na magistrali widocznego na oscyloskopie, lub jest on całkowicie niezrozumiały/zniekształcony, a debugowanie oprogramowania nie przynosi efektu.
 - **Przyczyna:** Magistrala CAN jest niezamknięta. **Brak rezystorów terminujących 120 Ω (Ohm) pomiędzy pinami CAN_H oraz CAN_L.** Sieć CAN działa w oparciu o różnicę napięć i wymaga fizycznego "ściągnięcia" linii do stanu recesywnego. Podczas testów "na biurku" upewnij się, że Twój sprzętowy analizator CAN ma wbudowany rezystor, lub dodaj go "na krótko" we wtyczce.
- **Objaw:** Transiwer CAN się nie odzywa.
 - **Przyczyna:** Pin `STBY` transiweru CAN nie jest ściągnięty do masy (GND). Zweryfikuj, czy odpowiednie rezystory konfiguracyjne zostały wlutowane na PCB.

ROS 2 / Linux

Pułapki Software'owe (ROS 2 / CMake / C++)

- **Objaw:** Kod kompiluje się z błędami `'std::span' has not been declared` lub `'concept' does not name a type` w systemie ROS 2.
 - **Rozwiązanie:** Wymuś standard C++20 w pliku `CMakeLists.txt` zaraz po dyrektywie `project`: `set(CMAKE_CXX_STANDARD 20)`. Pakiety ROS 2 (Humble) domyślnie używają starszego C++17.
- **Objaw:** Zmienne we własnym kodzie nie odpowiadają nazwom wygenerowanym przez generator `cantools` (błąd "has no member named...").
 - **Rozwiązanie:** Generator w Pythonie wymusza styl `snake_case`. Sygnały zdefiniowane w pliku `.dbc` jako `AMK_bSystemReady` zostają wygenerowane jako: `amk_b_system_ready`. Zawsze weryfikuj wygląd pól w wygenerowanym pliku `.h`.